



Universidad Autónoma de Chihuahua

Facultad de Ingeniería

Grupo 5k2

Actividad 1:
MongoDB Cluster

Bases de Datos Avanzadas

Docente: José Saúl De Lira Miramontes

Osvaldo Hernández Juarez
Matrícula: 385493

Diego Flores Verdad Grijalva
Matrícula: 385660

Luis Felipe Domínguez Chávez
Matrícula: 385500

29 de octubre de 2026

Índice

1 Introducción **2**

2 Objetivo **2**

3 Entorno de trabajo **2**

4 Configurar un Cluster en Mongo sobre Docker con los siguientes nodos: configsvr, shard1, shard2, shard3, mongos router **2**

4.1 Arquitectura 2
4.2 Docker Compose 3
4.3 Levantamiento de los contenedores 4
4.4 Inicialización de los replica sets 5
4.5 Registro de los shards 7

5 Defina la estructura de la base de datos **8**

6 Definir el Shard Key de la colección **8**

6.1 Carga de datos 11
6.2 Estado del cluster 13

7 Realizar operaciones CRUD sobre la base de datos distribuida **14**

7.1 Create 14
7.2 Read 16
7.3 Update 18
7.4 Delete 19
7.5 Distribución de los datos 19

8 Conclusión **20**

Referencias **21**

1 Introducción

En la actividad anterior se revisó en teoría la arquitectura de MongoDB como sistema distribuido. En esta práctica se puso en funcionamiento esa arquitectura: se configuró un Sharded Cluster sobre Docker con un config server, tres shards y un router mongos, se definió la estructura de una colección de usuarios y su shard key, y se realizaron operaciones CRUD sobre la base de datos distribuida a través del router.

Los archivos de la práctica (Docker Compose, scripts de configuración y operaciones CRUD) se encuentran en el repositorio https://github.com/LF-Dguez/MongoDB_cluster_sharding.

2 Objetivo

Implementar soluciones para Bases de Datos Distribuidas.

3 Entorno de trabajo

La práctica se realizó en Windows 11 con Docker Desktop y Docker Compose v2. Los cinco nodos usan la imagen oficial `mongo:8.0`. Los scripts de configuración y de CRUD se escribieron en JavaScript y se ejecutaron con el `mongosh` que incluye la imagen, dentro del contenedor del router.

4 Configurar un Cluster en Mongo sobre Docker con los siguientes nodos: configsvr, shard1, shard2, shard3, mongos router

4.1 Arquitectura

El cluster tiene tres tipos de nodo, cada uno en su propio contenedor:

- `configsvr`: guarda los metadatos del cluster, como la lista de shards y los rangos de datos (chunks) que tiene cada uno.
- `shard1`, `shard2` y `shard3`: guardan los documentos. Cada uno contiene solo una parte de la colección.
- `mongos`: es el router. Las aplicaciones se conectan a él, y él consulta al config server para saber a qué shard enviar cada operación. No guarda datos.

MongoDB exige que el config server y cada shard sean un replica set. Como la actividad pide exactamente cinco nodos, cada replica set tiene un solo miembro. En un entorno de producción cada uno tendría al menos tres para tener alta disponibilidad.

4.2 Docker Compose

Los cinco nodos se definieron en el archivo `docker-compose.yaml`:

```
services:
  configsvr:
    image: mongo:8.0
    container_name: configsvr
    command: mongod --configsvr --replSet configReplSet --port 27017 --
       bind_ip_all
    volumes:
      - configsvr_data:/data/configdb
    networks:
      - mongo-cluster

  shard1:
    image: mongo:8.0
    container_name: shard1
    command: mongod --shardsvr --replSet shard1ReplSet --port 27017 --
       bind_ip_all
    volumes:
      - shard1_data:/data/db
    networks:
      - mongo-cluster

  shard2:
    image: mongo:8.0
    container_name: shard2
    command: mongod --shardsvr --replSet shard2ReplSet --port 27017 --
       bind_ip_all
    volumes:
      - shard2_data:/data/db
    networks:
      - mongo-cluster

  shard3:
    image: mongo:8.0
    container_name: shard3
    command: mongod --shardsvr --replSet shard3ReplSet --port 27017 --
       bind_ip_all
    volumes:
      - shard3_data:/data/db
    networks:
      - mongo-cluster

  mongos:
    image: mongo:8.0
    container_name: mongos
    command: mongos --configdb configReplSet/configsvr:27017 --port 27017
       --bind_ip_all
    ports:
      - "27030:27017"
    volumes:
      - ./configuracion:/scripts/configuracion:ro
      - ./crud:/scripts/crud:ro
    depends_on:
      - configsvr
    networks:
```

```
      - mongo-cluster

volumes:
  configsvr_data:
  shard1_data:
  shard2_data:
  shard3_data:

networks:
  mongo-cluster:
```

Los puntos principales de la configuración son:

- El config server se inicia con `--configsvr` y los shards con `--shardsvr`. Cada uno recibe con `--replSet` el nombre de su replica set.
- Todos escuchan en el puerto 27017 dentro de su contenedor. La opción `--bind_ip_all` hace que acepten conexiones de los demás contenedores. Sin esa opción, cada `mongod` solo escucharía en su propio localhost.
- El router ejecuta el binario `mongos`, no `mongod`, y con `--configdb` se le indica dónde está el config server.
- Solo `mongos` se publica hacia el equipo, en el puerto 27030; los shards y el config server quedan ocultos, como en un cluster real. Se usó el 27030 porque el 27017 lo ocupa el MongoDB instalado en Windows.
- Los contenedores comparten la red `mongo-cluster`, en la que se encuentran por su nombre (`configsvr`, `shard1`, etc.).
- Cada nodo de datos tiene su volumen para que la información sobreviva si se recrea el contenedor. Las carpetas `configuracion` y `crud` del repositorio se montan en `mongos`, para ejecutar los scripts con el `mongosh` de ese contenedor.

4.3 Levantamiento de los contenedores

```
docker compose up -d
docker compose ps
```

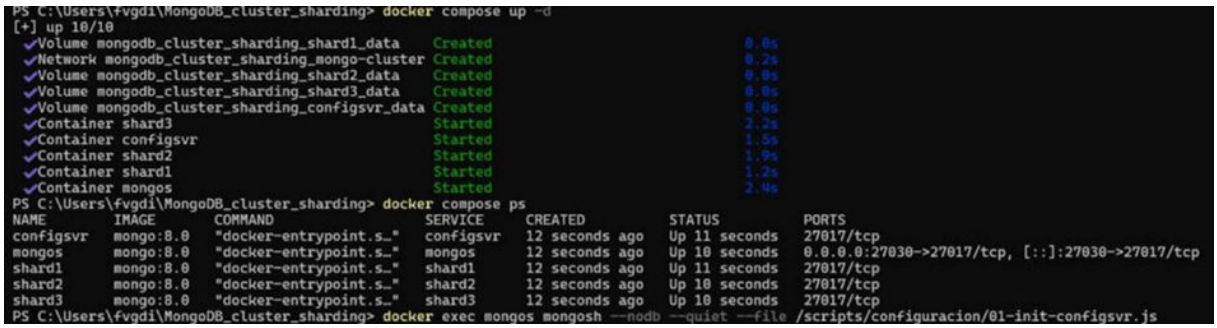


Figura 1: Creación y estado de los cinco contenedores.

Compose creó la red, los volúmenes y los cinco contenedores, que aparecen en ejecución. En este punto los nodos todavía no forman un cluster: hay que inicializar los replica sets y registrar los shards.

4.4 Inicialización de los replica sets

El replica set del config server se inicializa con el comando `replSetInitiate`. El script se ejecuta desde el contenedor de `mongos` con la opción `--nodb`, y abre la conexión al config server por su nombre en la red de Docker. El host del miembro se registra como `configsvr:27017` y no como `localhost`, porque esa dirección queda guardada en la configuración y es la que usan los demás nodos para conectarse.

```
// Paso 4a. Inicializa el replica set del config server.
//
// Se ejecuta desde el contenedor de mongos, donde estan montados los
   scripts:
// docker exec mongos mongosh --nodb --quiet
//   --file /scripts/configuracion/01-init-configsvr.js
//
// --nodb porque todavia no hay nada a que conectarse por defecto; la
   conexion
// al config server se abre aqui mismo por su nombre en la red de Docker.

const uri = "mongodb://configsvr:27017/?directConnection=true";
const admin = new Mongo(uri).getDB("admin");

const res = admin.runCommand({
  replSetInitiate: {
    _id: "configReplSet",
    configsvr: true,
    members: [{ _id: 0, host: "configsvr:27017" }]
  }
});

if (res.ok === 1) {
  print("configReplSet: replica set iniciado");
} else if (res.codeName === "AlreadyInitialized") {
  print("configReplSet: ya estaba iniciado");
} else {
  printjson(res);
  quit(1);
}

// La eleccion del PRIMARY tarda unos segundos.
for (let i = 0; i < 30; i++) {
  if (admin.runCommand({ hello: 1 }).isWritablePrimary) break;
  sleep(1000);
}

const estado = admin.runCommand({ replSetGetStatus: 1 });
estado.members.forEach(m => print(`  ${m.name}  ${m.stateStr}`));
```

```
docker exec mongos mongosh --nodb --quiet --file /scripts/configuracion/01-
```

```
    init-configsvr.js
```



Figura 2: Inicialización del replica set del config server.

El script espera a que termine la elección y muestra al único miembro como `PRIMARY`.

Para los shards el procedimiento es el mismo, con tres diferencias: el nombre del replica set, el host del miembro y que no lleva la opción `configsvr`.

```
// Paso 4b. Inicializa el replica set de cada shard.
//
// docker exec mongos mongosh --nodb --quiet
//   --file /scripts/configuracion/02-init-shards.js
//
// Es lo mismo que en el config server, con tres diferencias: el _id del
// replica set, el host del miembro y que no lleva configsvr: true.

const shards = [
  { rs: "shard1ReplSet", host: "shard1:27017" },
  { rs: "shard2ReplSet", host: "shard2:27017" },
  { rs: "shard3ReplSet", host: "shard3:27017" }
];

shards.forEach(s => {
  const uri = `mongodb://${s.host}/?directConnection=true`;
  const admin = new Mongo(uri).getDB("admin");

  const res = admin.runCommand({
    replSetInitiate: {
      _id: s.rs,
      members: [{ _id: 0, host: s.host }]
    }
  });

  if (res.ok === 1) {
    print(`${s.rs}: replica set iniciado`);
  } else if (res.codeName === "AlreadyInitialized") {
    print(`${s.rs}: ya estaba iniciado`);
  } else {
    printjson(res);
    quit(1);
  }

  for (let i = 0; i < 30; i++) {
    if (admin.runCommand({ hello: 1 }).isWritablePrimary) break;
    sleep(1000);
  }

  const estado = admin.runCommand({ replSetGetStatus: 1 });
  estado.members.forEach(m => print(`  ${m.name}  ${m.stateStr}`));
});
```

```
docker exec mongos mongosh --nodb --quiet --file /scripts/configuracion/02-
   init-shards.js
```

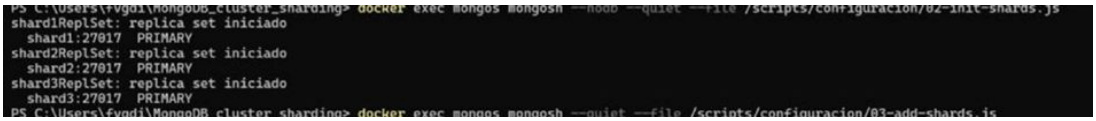


Figura 3: Inicialización de los replica sets de los tres shards.

Los tres shards quedaron con su miembro como `PRIMARY`.

4.5 Registro de los shards

Con los replica sets listos, los shards se registran desde `mongos` con `sh.addShard`. El registro se guarda en el config server, y a partir de ese momento el router sabe a qué nodos puede enviar los datos.

```
// Paso 5. Registra los tres shards en el cluster.
//
// docker exec mongos mongosh --quiet
//   --file /scripts/configuracion/03-add-shards.js
//
// Sin --nodb, mongosh se conecta al mongos del propio contenedor. El
   formato
// es "nombreReplicaSet/host:puerto". mongos guarda el registro en el
   config
// server, y desde ese momento sabe a que nodos puede mandar los datos.

const shards = [
  "shard1ReplSet/shard1:27017",
  "shard2ReplSet/shard2:27017",
  "shard3ReplSet/shard3:27017"
];

shards.forEach(cadena => {
  const res = sh.addShard(cadena);
  print(`addShard ${cadena}: ok ${res.ok}`);
});

print("\nShards registrados:");
db.adminCommand({ listShards: 1 }).shards
  .forEach(s => print(`  ${s._id}  ${s.host}`));
```

```
docker exec mongos mongosh --quiet --file /scripts/configuracion/03-add-
   shards.js
```



Figura 4: Registro de los tres shards en el cluster.

`listShards` confirma que el cluster tiene los tres shards, cada uno identificado por su replica set.

5 Defina la estructura de la base de datos

Se eligió una base de datos `bdcluster` con una colección `usuarios`. Cada documento representa a un usuario con datos personales, una dirección como subdocumento y una lista de intereses:

```
{
  idUsuario: 1,
  nombre: "Ana",
  apellido: "Torres",
  email: "ana.torres1@correo.com",
  edad: 24,
  direccion: { ciudad: "Chihuahua", estado: "Chihuahua", pais: "México" },
  intereses: ["música", "cine"],
  fechaRegistro: ISODate("2025-03-14"),
  activo: true
}
```

La colección se creó con un validador `$jsonSchema` (sección 6) que exige los campos idUsuario, nombre, apellido, email, fechaRegistro y activo, y revisa el tipo de cada campo. Así, aunque MongoDB no obliga a que los documentos tengan la misma estructura, todos los usuarios que se inserten respetan este formato.

6 Definir el Shard Key de la colección

La shard key elegida fue `{ idUsuario: "hashed"}`. Los criterios fueron los siguientes:

- Cardinalidad: idUsuario es único, así que tiene tantos valores distintos como usuarios y los datos se pueden dividir en muchos chunks.
- Frecuencia: ningún valor se repite, por lo que no hay chunks que crezcan sin poder dividirse.
- Crecimiento monótono: idUsuario aumenta con cada registro. Con sharding por rangos, todas las inserciones nuevas caerían en el último chunk y en un solo shard. Al aplicar un hash, los valores consecutivos quedan repartidos entre los tres shards.

- Consultas: las operaciones sobre un usuario se hacen por su idUsuario, y con esa condición `mongos` envía la operación a un solo shard.

La desventaja del hash es que las consultas por rango de idUsuario tienen que ir a todos los shards, pero en esta colección ese tipo de consulta no es la más común. Además, al fragmentar una colección vacía con una shard key hashed, MongoDB reparte los chunks entre los shards desde el inicio. Con sharding por rangos, la colección empieza en un solo shard y el balanceador solo mueve datos cuando la diferencia entre shards es grande, lo que con pocos miles de documentos no ocurriría.

```
// Pasos 6 y 7a. Crea la coleccion usuarios con su estructura y la
   fragmenta.
//
// docker exec mongos mongosh --quiet
//   --file /scripts/configuracion/04-shard-collection.js
//
// Estructura de un documento de usuarios:
// {
//   idUsuario: 1,                  // entero, unico (shard key)
//   nombre: "Ana",
//   apellido: "Torres",
//   email: "ana.torres1@correo.com",
//   edad: 24,
//   direccion: { ciudad: "Chihuahua", estado: "Chihuahua", pais: "México"
   },
//   intereses: ["música", "cine"],
//   fechaRegistro: ISODate("2025-03-14"),
//   activo: true
// }
//
// Shard key: { idUsuario: "hashed" }
//  - Cardinalidad alta: cada usuario tiene un id distinto.
//  - idUsuario crece con cada registro; por rangos, todas las inserciones
//    nuevas caerian en el ultimo chunk. El hash lo evita.
//  - Con hash, MongoDB reparte la coleccion vacia entre los tres shards
//    desde el inicio, aunque haya pocos documentos.
//  - Buscar, actualizar o borrar por idUsuario va a un solo shard.

const bd = db.getSiblingDB("bdcluster");

bd.usuarios.drop();

bd.createCollection("usuarios", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["idUsuario", "nombre", "apellido", "email",
                 "fechaRegistro", "activo"],
      properties: {
        idUsuario: { bsonType: "number" },
        nombre: { bsonType: "string" },
        apellido: { bsonType: "string" },
        email: { bsonType: "string", pattern: "^.+@.+$" },
        edad: { bsonType: "number", minimum: 0 },
        direccion: {
```

```
          bsonType: "object",
          properties: {
            ciudad: { bsonType: "string" },
            estado: { bsonType: "string" },
            pais: { bsonType: "string" }
          }
        },
        intereses: { bsonType: "array", items: { bsonType: "string" } },
        fechaRegistro: { bsonType: "date" },
        activo: { bsonType: "bool" }
      }
    }
  }
});
print("Coleccion bdcluster.usuarios creada con validador");

printjson(sh.enableSharding("bdcluster"));
printjson(sh.shardCollection("bdcluster.usuarios", { idUsuario: "hashed" })
    );

// Chunks que quedaron en cada shard antes de insertar datos
print("\nChunks por shard:");
db.getSiblingDB("config").chunks.aggregate([
  { $lookup: { from: "collections", localField: "uuid",
               foreignField: "uuid", as: "col" } },
  { $match: { "col._id": "bdcluster.usuarios" } },
  { $group: { _id: "$shard", chunks: { $sum: 1 } } },
  { $sort: { _id: 1 } }
]).forEach(c => print(`  ${c._id}: ${c.chunks}`));
```

```
docker exec mongos mongosh --quiet --file /scripts/configuracion/04-shard-
   collection.js
```

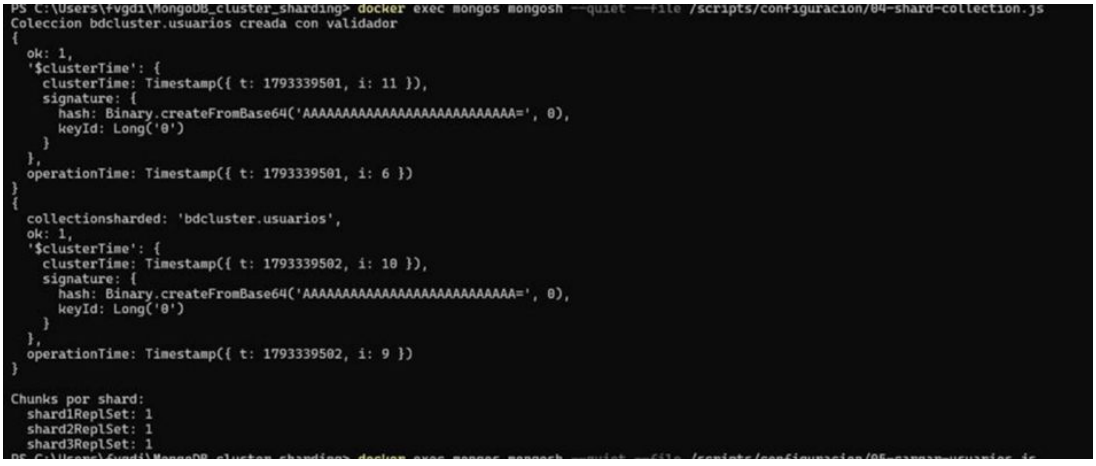


Figura 5: Creación y fragmentación de la colección usuarios.

La colección quedó fragmentada y, antes de insertar cualquier documento, cada shard ya tiene asignados sus chunks.

6.1 Carga de datos

Para tener datos sobre los cuales probar la distribución, se cargaron 3000 usuarios generados a partir de listas de nombres, apellidos, ciudades e intereses. El generador usa una semilla fija, de modo que siempre produce los mismos documentos.

```
// Paso 7b. Carga inicial de usuarios a traves de mongos.
//
// docker exec mongos mongosh --quiet
//   --file /scripts/configuracion/05-cargar-usuarios.js
//
// Genera 3000 usuarios con datos de ejemplo. Usa un generador con semilla
// fija para que cada vez que se corra salgan los mismos datos.

const TOTAL = 3000;
const bd = db.getSiblingDB("bdcluster");

let semilla = 2026;
function azar(n) {
  semilla = (semilla * 16807) % 2147483647;
  return semilla % n;
}
const elegir = lista => lista[azar(lista.length)];

const nombres = [
  "Ana", "Luis", "María", "Carlos", "Sofía", "Jorge", "Valeria",
  "Diego", "Fernanda", "Miguel", "Daniela", "José", "Camila", "Andrés",
  "Paola", "Ricardo", "Lucía", "Fernando", "Gabriela", "Raúl"
];
const apellidos = [
  "García", "Hernández", "Martínez", "López", "González", "Pérez",
  "Rodríguez", "Sánchez", "Ramírez", "Torres", "Flores", "Rivera",
  "Gómez", "Díaz", "Cruz", "Morales", "Reyes", "Ortiz", "Chávez",
  "Domínguez"
];
const ubicaciones = [
  { ciudad: "Chihuahua", estado: "Chihuahua", pais: "México" },
  { ciudad: "Ciudad Juárez", estado: "Chihuahua", pais: "México" },
  { ciudad: "Monterrey", estado: "Nuevo León", pais: "México" },
  { ciudad: "Guadalajara", estado: "Jalisco", pais: "México" },
  { ciudad: "Ciudad de México", estado: "CDMX", pais: "México" },
  { ciudad: "Puebla", estado: "Puebla", pais: "México" },
  { ciudad: "El Paso", estado: "Texas", pais: "Estados Unidos" },
  { ciudad: "Bogotá", estado: "Cundinamarca", pais: "Colombia" },
  { ciudad: "Madrid", estado: "Madrid", pais: "España" },
  { ciudad: "Buenos Aires", estado: "Buenos Aires", pais: "Argentina" }
];
const temas = [
  "música", "cine", "deportes", "videojuegos", "lectura", "viajes",
  "programación", "cocina", "fotografía", "arte"
];

// Para el email: minusculas y sin acentos
const sinAcentos = t => t.toLowerCase()
  .replace(/[áéíóú]/g, c => "aeiou"["áéíóú".indexOf(c)]);

const inicio = new Date("2023-01-01").getTime();
```

```
const dias = 1370; // hasta octubre de 2026

const usuarios = [];
for (let id = 1; id <= TOTAL; id++) {
  const nombre = elegir(nombres);
  const apellido = elegir(apellidos);
  usuarios.push({
    idUsuario: id,
    nombre: nombre,
    apellido: apellido,
    email: `${sinAcentos(nombre)}.${sinAcentos(apellido)}${id}@correo.com`,
    edad: 18 + azar(53),
    direccion: elegir(ubicaciones),
    intereses: [...new Set([elegir(temas), elegir(temas), elegir(temas)])],
    fechaRegistro: new Date(inicio + azar(dias) * 86400000),
    activo: azar(10) < 8
  });
}

bd.usuarios.deleteMany({});
for (let i = 0; i < usuarios.length; i += 1000) {
  bd.usuarios.insertMany(usuarios.slice(i, i + 1000));
}

print(`Usuarios en bdcluster.usuarios: ${bd.usuarios.countDocuments()}`);
print("\nDocumentos por shard:");
bd.usuarios.aggregate([{ $collStats: { count: {} } }, { $sort: { shard: 1 } }])
  .forEach(s => print(`  ${s.shard}: ${s.count}`));
```

```
docker exec mongos mongosh --quiet --file /scripts/configuracion/05-cargar-
   usuarios.js
```



Figura 6: Carga de los usuarios y documentos en cada shard.

Los documentos se insertaron a través de `mongos`, que los envió a cada shard según el hash de su idUsuario. La etapa `$collStats` muestra cuántos documentos quedaron en cada uno.

6.2 Estado del cluster

```
docker exec -it mongos mongosh --eval "sh.status()"
```



Figura 7: Estado del cluster con `sh.status()`.

El resultado de `sh.status()` resume la configuración completa: los tres shards,

la base `bdcluster` y la colección `usuarios`, con su shard key y el número de chunks en cada shard.

7 Realizar operaciones CRUD sobre la base de datos distribuida

Las operaciones CRUD se hicieron con `mongosh` conectado a `mongos`, sobre la base `bdcluster`. Al pasar por el router, cada operación se enruta de forma transparente al shard (o shards) correspondiente sin que el script tenga que saber en cuál vive cada documento.

7.1 Create

Se insertó un usuario con `insertOne` y tres más con `insertMany`, todos con `idUsuario` mayor a 3000 para no chocar con la carga inicial. También se intentó insertar un documento incompleto para comprobar que el validador `$jsonSchema` de la colección lo rechaza.

```
// Paso 8 (Diego). Create: insertOne e insertMany sobre bdcluster.usuarios.
//
// docker exec mongos mongosh bdcluster --quiet --file /scripts/crud/create
    .js
//
// La carga inicial (05-cargar-usuarios.js) uso los ids 1 a 3000, asi que
// aqui se insertan ids mayores a 3000 para no chocar con esos documentos.

print("--- insertOne ---");
const resultadoUno = db.usuarios.insertOne({
  idUsuario: 3001,
  nombre: "Renata",
  apellido: "Salcido",
  email: "renata.salcido3001@correo.com",
  edad: 29,
  direccion: { ciudad: "Chihuahua", estado: "Chihuahua", pais: "México" },
  intereses: ["fotografía", "viajes"],
  fechaRegistro: new Date("2026-10-29"),
  activo: true
});
printjson(resultadoUno);

print("\n--- insertMany ---");
const resultadoVarios = db.usuarios.insertMany([
  {
    idUsuario: 3002,
    nombre: "Emiliano",
    apellido: "Bustillos",
    email: "emiliano.bustillos3002@correo.com",
    edad: 34,
    direccion: { ciudad: "Monterrey", estado: "Nuevo León", pais: "México"
       },
    intereses: ["deportes", "música"],
    fechaRegistro: new Date("2026-10-29"),
    activo: true
```

```
  },
  {
    idUsuario: 3003,
    nombre: "Ximena",
    apellido: "Quiñones",
    email: "ximena.quinones3003@correo.com",
    edad: 22,
    direccion: { ciudad: "Guadalajara", estado: "Jalisco", pais: "México"
       },
    intereses: ["cine", "lectura"],
    fechaRegistro: new Date("2026-10-29"),
    activo: false
  },
  {
    idUsuario: 3004,
    nombre: "Tomás",
    apellido: "Nevárez",
    email: "tomas.nevarez3004@correo.com",
    edad: 41,
    direccion: { ciudad: "Bogotá", estado: "Cundinamarca", pais: "Colombia"
        },
    intereses: ["programación"],
    fechaRegistro: new Date("2026-10-29"),
    activo: true
  }
]);
printjson(resultadoVarios);

print("\n--- Insert invalido (debe fallar el validador) ---");
try {
  db.usuarios.insertOne({
    idUsuario: 3005,
    nombre: "Sin apellido ni email"
    // Faltan apellido, email, fechaRegistro y activo: el $jsonSchema lo
       rechaza.
  });
} catch (error) {
  print(`Rechazado por el validador: ${error.message}`);
}

print(`\nTotal de documentos en bdcluster.usuarios: ${db.usuarios.
   countDocuments()}`);
```

```
docker exec mongos mongosh bdcluster --quiet --file /scripts/crud/create.js
```



Figura 8: Inserción de usuarios a través de mongos.

Las dos inserciones quedaron reconocidas (`acknowledged: true`) y el documento incompleto fue rechazado con el error de validación, por lo que el total pasó de 3000 a 3004 usuarios.

7.2 Read

Se hicieron dos tipos de consulta: una por `idUsuario` (la shard key) y otras por `direccion.pais` e `intereses`, que no forman parte de la shard key.

```
// Paso 8 (Diego). Read: consultas por idUsuario (shard key) y por otros
// campos, mas explain() para comparar como enruta mongos cada una.
//
// docker exec mongos mongosh bdcluster --quiet --file /scripts/crud/read.
   js

print("--- find por idUsuario (va a un solo shard) ---");
printjson(db.usuarios.findOne({ idUsuario: 1500 }));

print("\n--- find por direccion.pais (va a todos los shards) ---");
db.usuarios.find({ "direccion.pais": "Colombia" }).limit(3).forEach(
   printjson);
print(`Total con direccion.pais "Colombia": ${db.usuarios.countDocuments({
   "direccion.pais": "Colombia" })}`);

print("\n--- find por intereses (va a todos los shards) ---");
print(`Total con "cine" en intereses: ${db.usuarios.countDocuments({
   intereses: "cine" })}`);

print("\n--- explain: find por idUsuario ---");
const explainPorId = db.usuarios.find({ idUsuario: 1500 }).explain("
   executionStats");
print(`winningPlan.stage: ${explainPorId.queryPlanner.winningPlan.stage}`);
printjson(Object.keys(explainPorId.queryPlanner.winningPlan.shards ?? {}));
print(`nReturned: ${explainPorId.executionStats.nReturned}`);

print("\n--- explain: find por direccion.pais ---");
const explainPorPais = db.usuarios.find({ "direccion.pais": "Colombia" }).
   explain("executionStats");
print(`winningPlan.stage: ${explainPorPais.queryPlanner.winningPlan.stage}`
   );
print(`nReturned: ${explainPorPais.executionStats.nReturned}`);
```

```
docker exec mongos mongosh bdcluster --quiet --file /scripts/crud/read.js
```

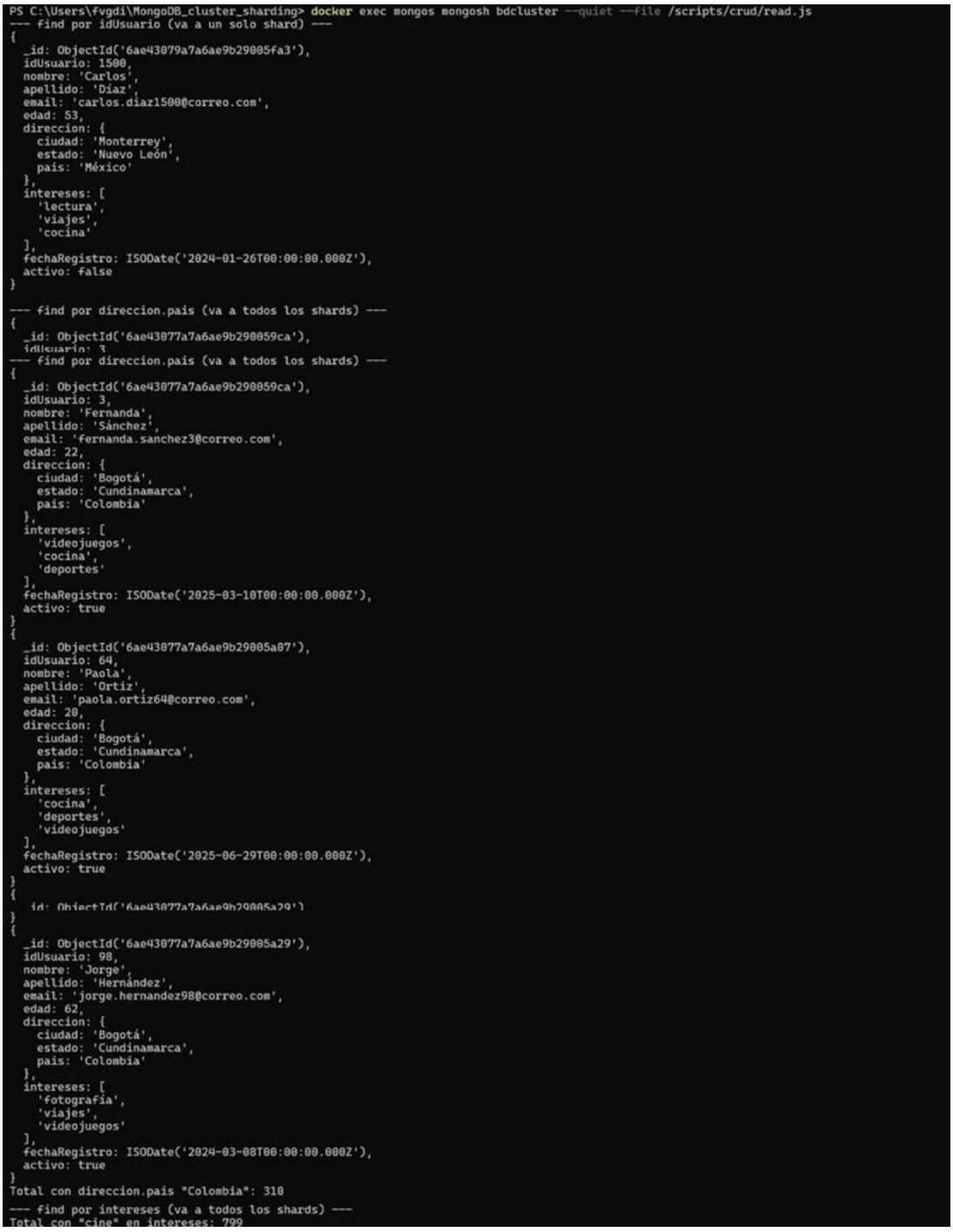


Figura 9: Consultas sobre la colección usuarios.



Figura 10: Plan de ejecución de una consulta por idUsuario y de una consulta por otro campo.

El `explain()` confirma la diferencia esperada: la consulta por `idUsuario` resuelve con `winningPlan.stage: "SINGLE_SHARD"`, porque `mongos` calcula el hash

y sabe en qué shard buscar, mientras que la consulta por `direccion.pais` resuelve con `"SHARD_MERGE"`, porque el router tiene que preguntarle a los tres shards y combinar los resultados.

7.3 Update

Se actualizó un usuario con `updateOne` (por `idUsuario`) y varios con `updateMany` (por `direccion.pais`).

```
// Paso 8 (Diego). Update: updateOne (por idUsuario) y updateMany.
//
// docker exec mongos mongosh bdcluster --quiet --file /scripts/crud/update
    .js

print("--- updateOne por idUsuario ---");
const resultadoUno = db.usuarios.updateOne(
  { idUsuario: 1500 },
  { $set: { edad: 99, activo: true } }
);
printjson(resultadoUno);
printjson(db.usuarios.findOne({ idUsuario: 1500 }));

print("\n--- updateMany ---");
const resultadoVarios = db.usuarios.updateMany(
  { "direccion.pais": "Colombia" },
  { $set: { activo: false } }
);
printjson(resultadoVarios);
print(`Inactivos en Colombia despues del update: ${db.usuarios.
    countDocuments({ "direccion.pais": "Colombia", activo: false })}`);
```

```
docker exec mongos mongosh bdcluster --quiet --file /scripts/crud/update.js
```

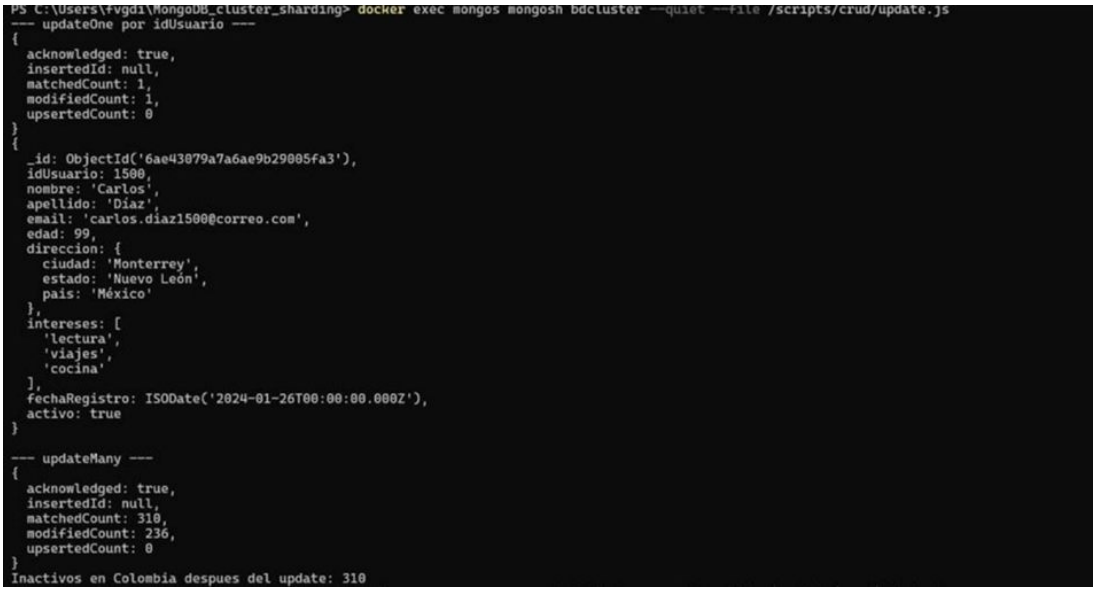


Figura 11: Actualización de usuarios.

El `updateOne` modificó el único documento con ese `idUsuario` (`matchedCount: 1`, `modifiedCount: 1`). El `updateMany` encontró 310 usuarios con `direccion.pais: Çolombia"` y modificó 236; los otros 74 ya tenían el valor que se quería asignar, así que no contaron como modificados.

7.4 Delete

Se borró un usuario con `deleteOne` (por `idUsuario`) y el resto de los usuarios insertados en `create.js` con `deleteMany`.

```
// Paso 8 (Diego). Delete: deleteOne (por idUsuario) y deleteMany.
//
// docker exec mongos mongosh bdcluster --quiet --file /scripts/crud/delete
   .js

print(`Total antes de borrar: ${db.usuarios.countDocuments()}`);

print("\n--- deleteOne por idUsuario ---");
const resultadoUno = db.usuarios.deleteOne({ idUsuario: 3001 });
printjson(resultadoUno);

print("\n--- deleteMany (resto de los usuarios insertados en create.js) ---
   ");
const resultadoVarios = db.usuarios.deleteMany({ idUsuario: { $gt: 3000 }
   });
printjson(resultadoVarios);

print(`\nTotal despues de borrar: ${db.usuarios.countDocuments()}`);
```

```
docker exec mongos mongosh bdcluster --quiet --file /scripts/crud/delete.js
```



Figura 12: Eliminación de usuarios.

El `deleteOne` borró un documento y el `deleteMany` los tres restantes, con lo que la colección regresó a los 3000 usuarios de la carga inicial.

7.5 Distribución de los datos

```
docker exec mongos mongosh bdcluster --quiet --eval "db.usuarios.
   getShardDistribution()"
```

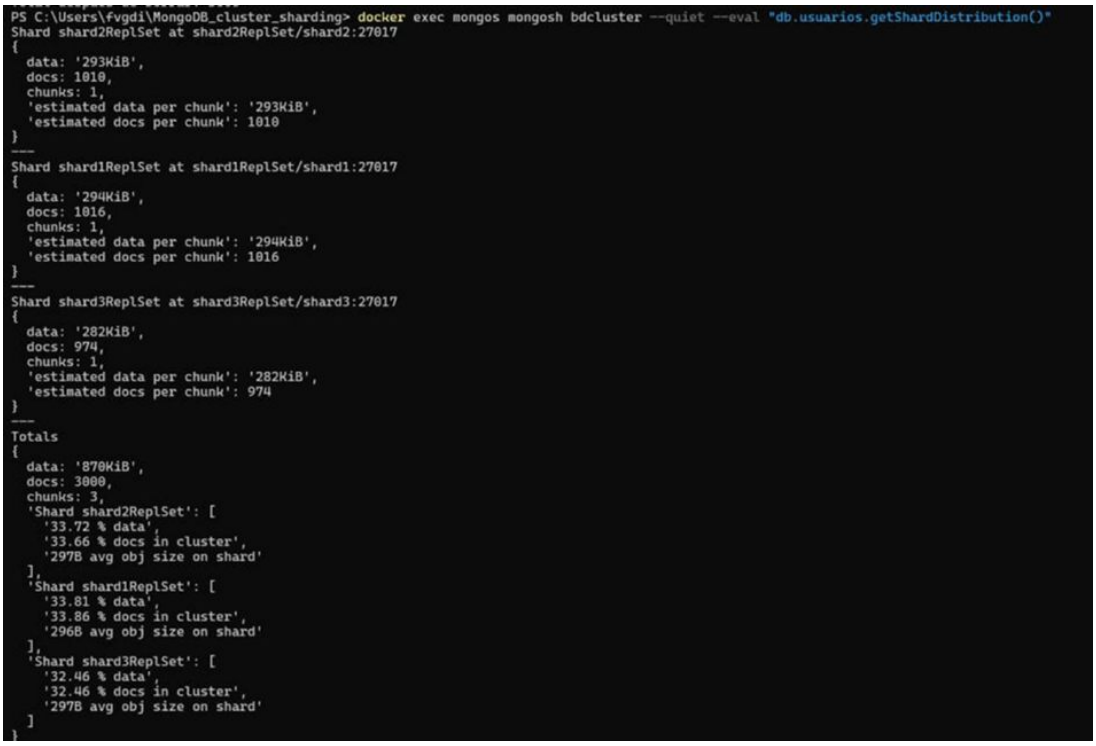


Figura 13: Distribución de los documentos entre los shards.

Después de las operaciones CRUD, los 3000 documentos siguen repartidos de forma pareja entre los tres shards (alrededor del 33 % en cada uno), lo mismo que después de la carga inicial, porque las inserciones y eliminaciones de esta sección se cancelaron entre sí.

8 Conclusión

Configurar el cluster mostró que el sharding en MongoDB no depende de un solo servidor sino de la coordinación entre componentes con funciones distintas: los shards guardan los datos, el config server sabe dónde está cada parte y el router dirige las operaciones. Para la aplicación, en cambio, todo eso es transparente, porque se conecta a `mongos` como si fuera un servidor normal.

La decisión más importante fue la shard key. Un campo con muchos valores distintos que aparece en las consultas más frecuentes permite que el router envíe cada operación al shard correcto, y aplicar un hash evita que un campo creciente concentre las inserciones en un solo shard.

En las operaciones CRUD esa transparencia se confirmó: los mismos comandos de `insertOne`, `find`, `updateOne` y `deleteOne` que se usarían contra un solo servidor funcionaron igual contra el cluster. La diferencia solo se hizo visible con `explain()`, al comparar una consulta por la shard key (un solo shard) contra una por otro campo (los tres shards).

Referencias

Borucki, A. (2025). Using replication and sharding. En *MongoDB 8.0 in Action* (3a ed., cap. 9). Manning.

Harsh. (2024). *Setting up MongoDB cluster: Replication, sharding, and high availability*. Medium.

MongoDB, Inc. (s.f.). *Sharding*. MongoDB Manual. https://www.mongodb.com/docs/manual/sharding/

udaykirankavaturu. (s.f.). *sharding-demo* [Repositorio]. GitHub. https://github.com/udaykirankavaturu/sharding-demo